🗺️ Simultaneous Mapping and Navigation (Synchronous Mapping Navigation)

1. Course Content

- Master the integrated working mechanism of "mapping while navigating" (Online SLAM + Nav2) in completely unknown environments.
- No need to remotely control the robot to pre-map the area; directly send navigation goal points in unknown environments, and the robot autonomously performs boundary exploration mapping, dynamic path planning, and real-time local obstacle avoidance.
- Master the dynamic costmap (Global Costmap) and online SLAM map dynamic linkage update mechanism in synchronous mapping navigation mode.
- Master the method for saving 2D occupancy raster map and pose graph (PoseGraph) after exploration mapping is complete.

---

2. Synchronous Mapping Navigation Principle Analysis and Advantages

In many sudden exploration tasks, detection inspections, or large-scale unknown scenarios, artificially remotely controlling the robot to traverse the entire site for mapping is often impractical. "Simultaneous Mapping & Navigation" deeply integrates online SLAM algorithm with Nav2 autonomous navigation protocol stack:

1. **Cold Start Without Prior**:
   The system starts online SLAM (SLAM-Toolbox) at the robot's current power-on position as origin `(0, 0, 0)`, real-time outputs initial local map and publishes authoritative `map -> odom` TF coordinate transformation.
2. **Unknown Area Path Planning (Allow Unknown Planning)**:
   By configuring Nav2 global planner (NavfnPlanner) to allow paths through unknown grids (`allow_unknown: true`), the robot can directly send navigation goal points to as-yet unexplored unknown gray areas and generate feasible exploration paths.
3. **Dynamic Costmap Linkage Refresh (Dynamic Costmap Sync)**:
   SLAM node high-frequency publishes the latest occupancy raster map to `/map` topic; the global costmap (Global Costmap) static layer real-time subscribes for updates, dynamically refreshing unknown areas to explored traversable areas or obstacles.
4. **Real-time Local Obstacle Avoidance Control (TEB Local Planner)**:
   Local planner, based on 235° forward lidar real-time ranging data, dynamically adjusts Ackermann front-wheel steering angle and drive speed, safely avoiding newly discovered obstacles on the travel route.

---

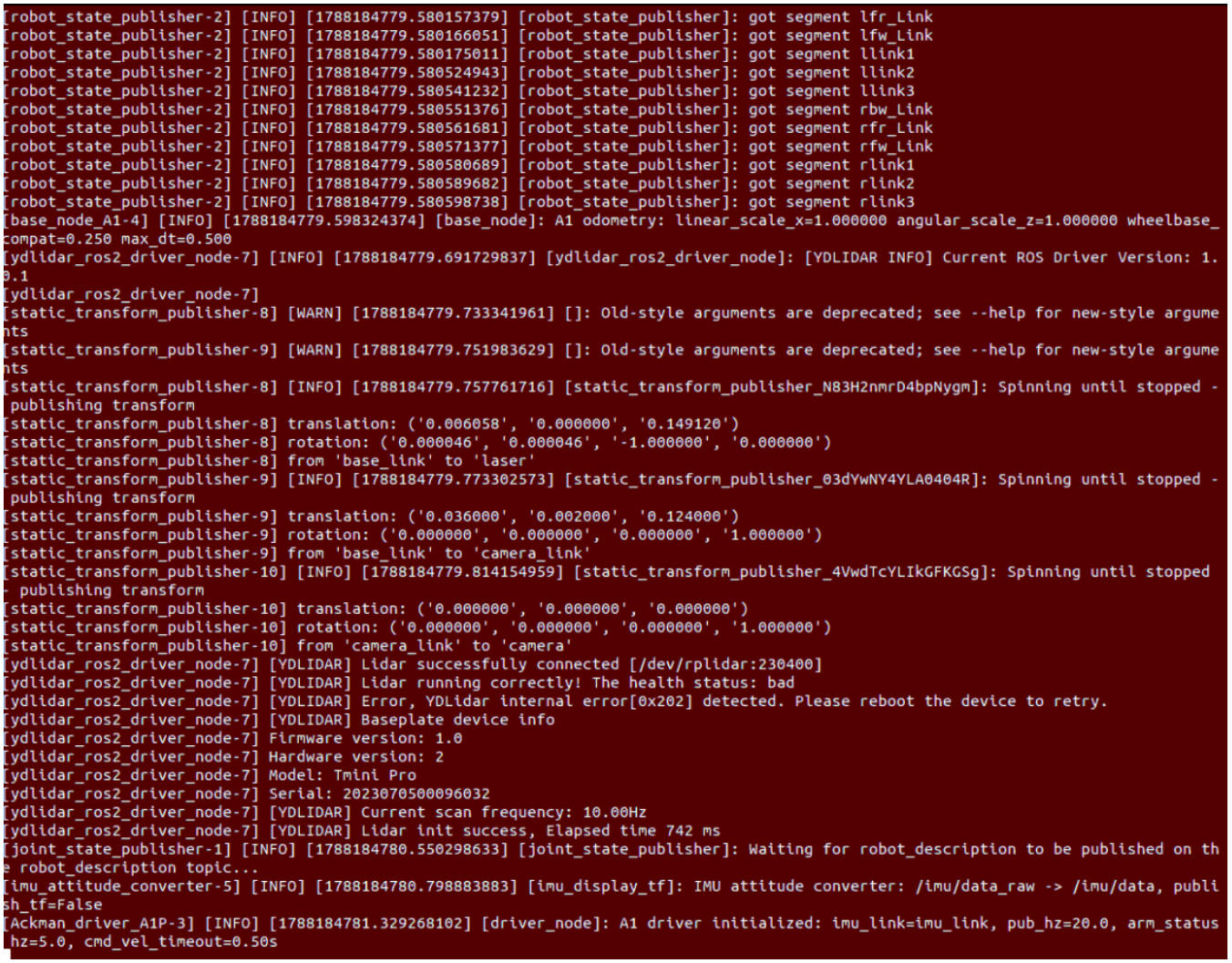
3. Preparation and Running

This example requires opening 3~4 terminals to start the following nodes in sequence:

3.1 Start Chassis + Lidar

Run in **Terminal 1** (start A1 Pro chassis driver, wheel odometry, and lidar driver):

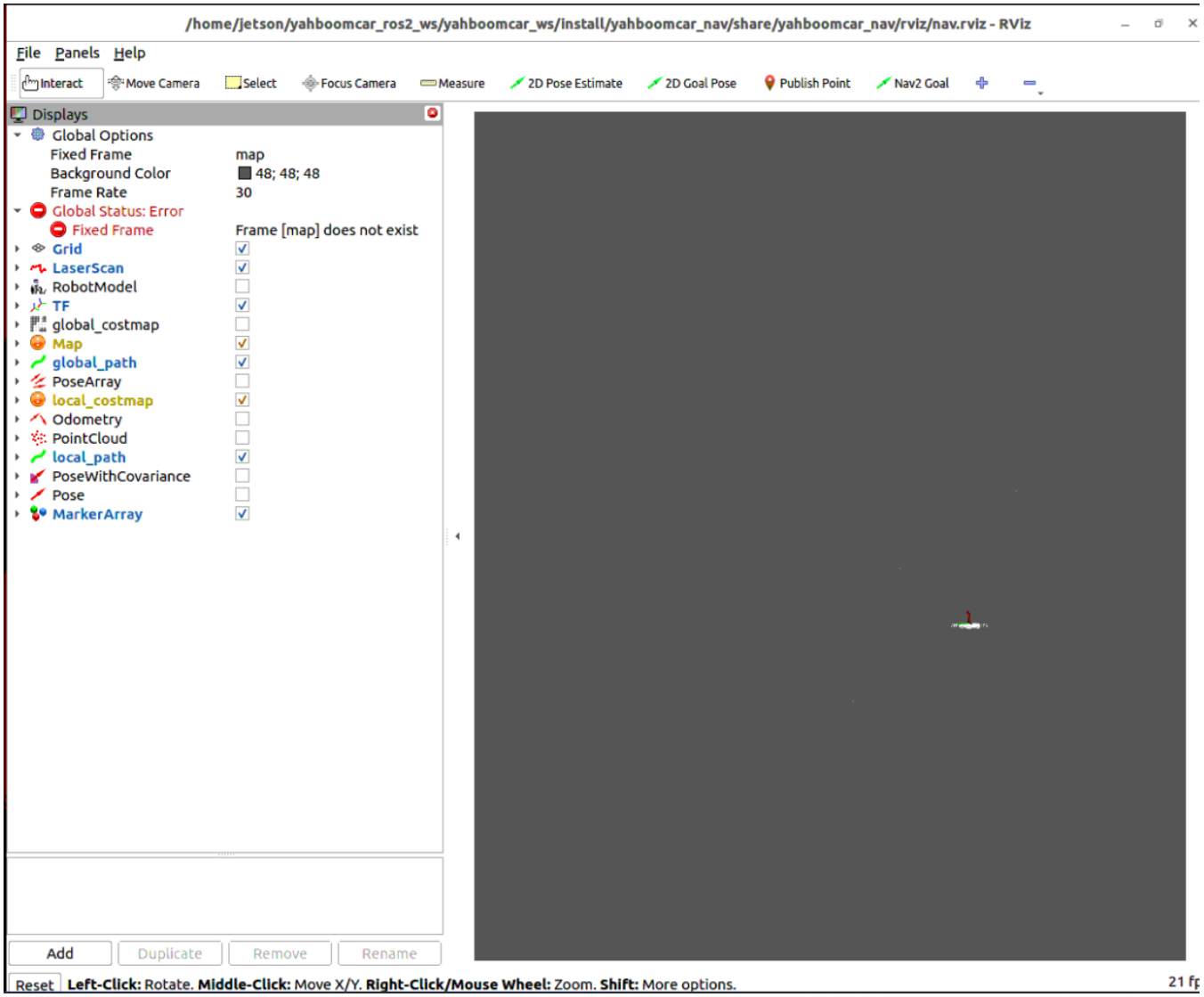
```
ros2 launch yahboomcar_nav laser_bringup_launch.py
```



3.2 Start RViz Navigation Visualization

Run in **Terminal 2** (it is recommended to open RViz on the virtual machine if the VM and robot are on the same LAN):

```
ros2 launch yahboomcar_nav display_nav_launch.py
```

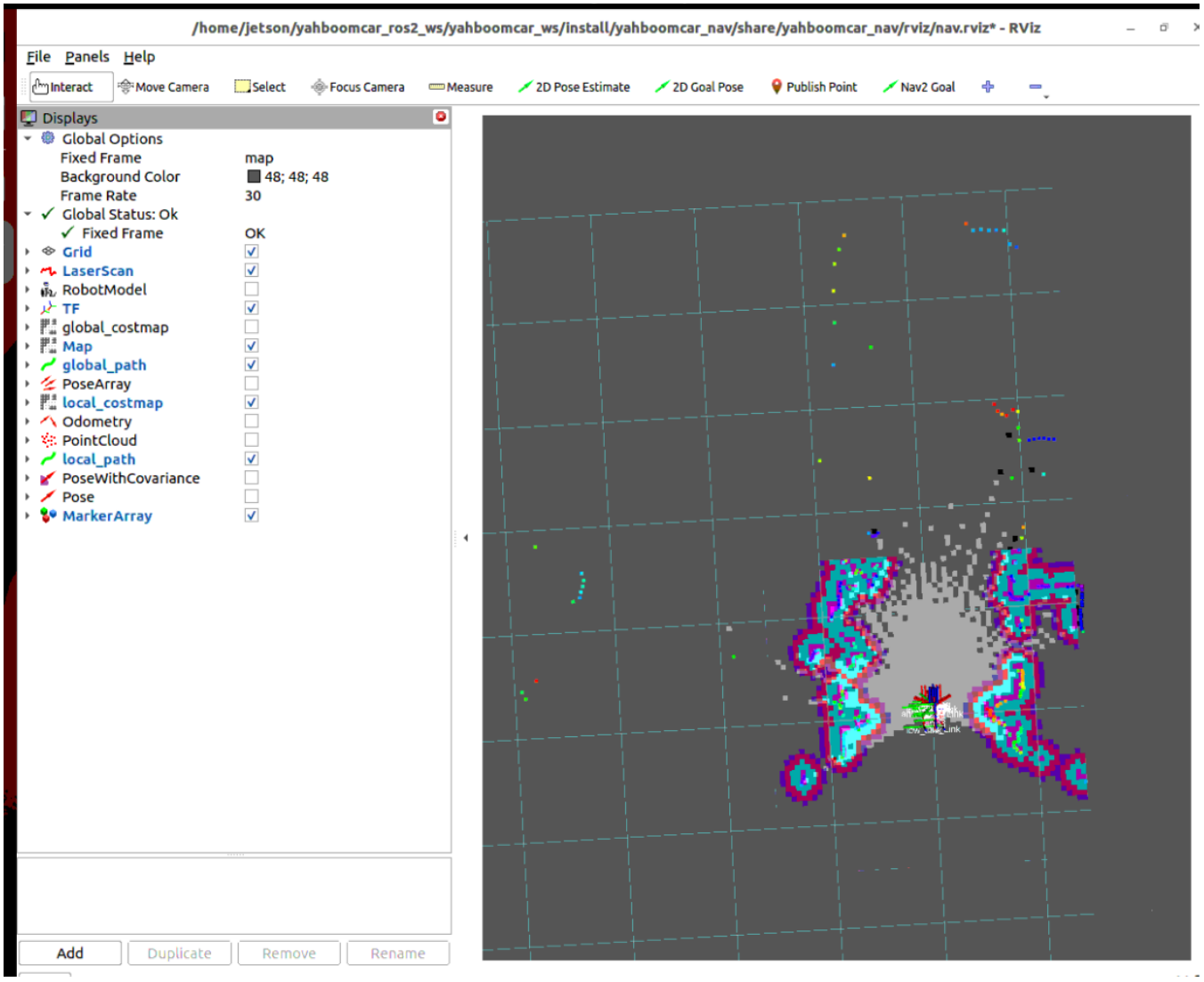


3.3 Start Synchronous Mapping Navigation Core

Run in **Terminal 3** with `slam:=True` to start (one-click pull up online SLAM and Nav2 navigation framework):

```
ros2 launch yahboomcar_nav navigation_slamtoolx_launch.py slam:=True
```

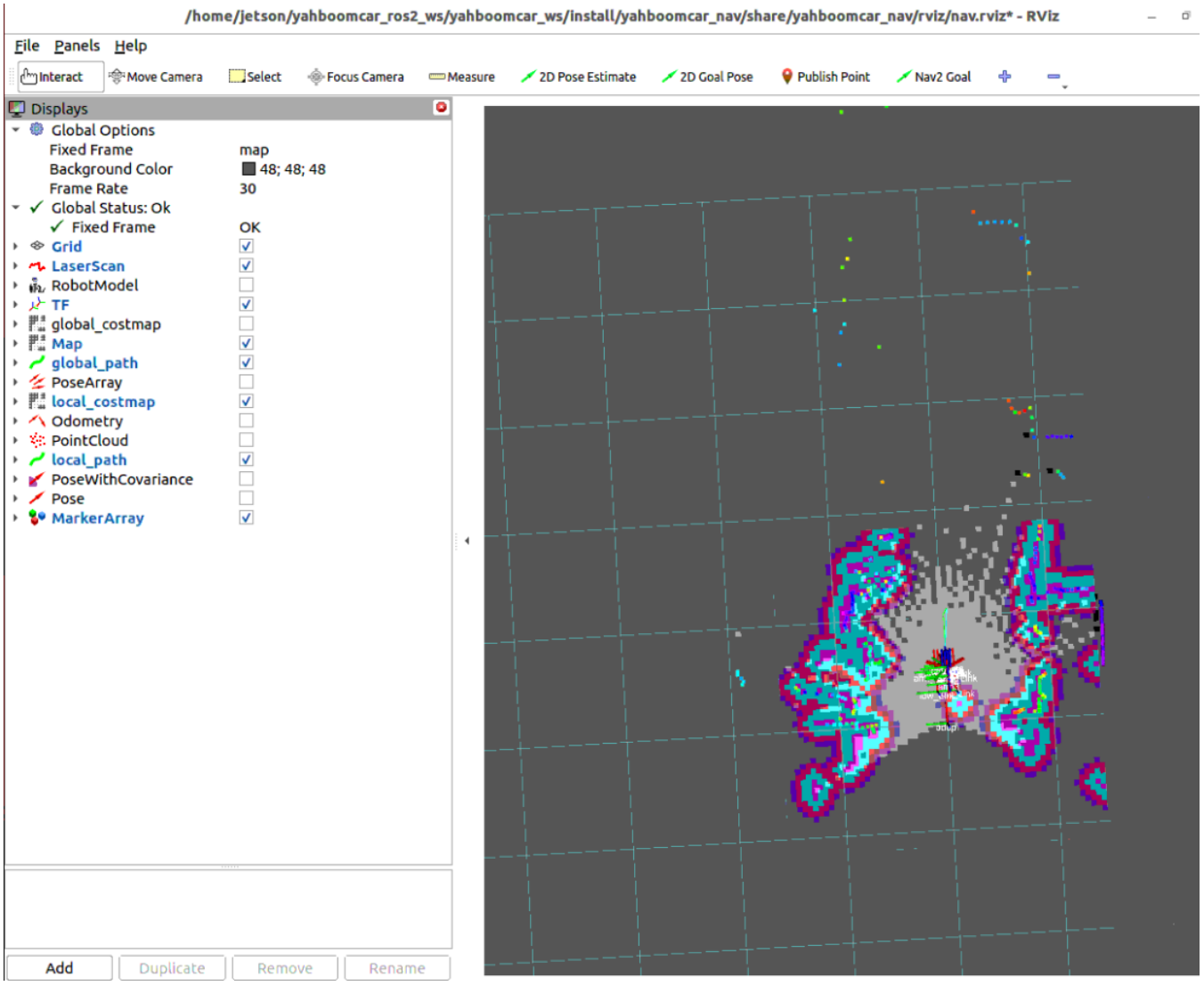
```
jetson@yahboom:~/yahboomcar_ros2_ws/yahboomcar_ws$ ros2 launch yahboomcar_nav navigation_slamtoolx_launch.py slam:=True
[component_container_isolated-1] [INFO] [1788184815.289343331] [controller_server]: Creating bond (controller_server) to lifecycle manager.
[component_container_isolated-1] [INFO] [1788184815.399203126] [lifecycle_manager_navigation]: Server controller_server connected with bond.
[component_container_isolated-1] [INFO] [1788184815.399329723] [lifecycle_manager_navigation]: Activating smoother_server
[component_container_isolated-1] [INFO] [1788184815.400534884] [smoother_server]: Activating
[component_container_isolated-1] [INFO] [1788184815.400639400] [smoother_server]: Creating bond (smoother_server) to lifecycle manager.
[component_container_isolated-1] [INFO] [1788184815.509977930] [lifecycle_manager_navigation]: Server smoother_server connected with bond.
[component_container_isolated-1] [INFO] [1788184815.510090573] [lifecycle_manager_navigation]: Activating planner_server
[component_container_isolated-1] [INFO] [1788184815.510531037] [planner_server]: Activating
[component_container_isolated-1] [INFO] [1788184815.510639520] [global_costmap.global_costmap]: Activating
[component_container_isolated-1] [INFO] [1788184815.510674817] [global_costmap.global_costmap]: Checking transform
[component_container_isolated-1] [INFO] [1788184815.511202932] [global_costmap.global_costmap]: start
[component_container_isolated-1] [INFO] [1788184815.562983202] [planner_server]: Activating plugin GridBased of type NavfnPlanner
[component_container_isolated-1] [INFO] [1788184815.564231756] [planner_server]: Creating bond (planner_server) to lifecycle manager.
[component_container_isolated-1] [INFO] [1788184815.670626537] [lifecycle_manager_navigation]: Server planner_server connected with bond.
[component_container_isolated-1] [INFO] [1788184815.670753902] [lifecycle_manager_navigation]: Activating behavior_server
[component_container_isolated-1] [INFO] [1788184815.672589933] [behavior_server]: Activating
[component_container_isolated-1] [INFO] [1788184815.672654447] [behavior_server]: Activating backup
[component_container_isolated-1] [INFO] [1788184815.672710929] [behavior_server]: Creating bond (behavior_server) to lifecycle manager.
[component_container_isolated-1] [INFO] [1788184815.780622306] [lifecycle_manager_navigation]: Server behavior_server connected with bond.
[component_container_isolated-1] [INFO] [1788184815.780750470] [lifecycle_manager_navigation]: Activating bt_navigator
[component_container_isolated-1] [INFO] [1788184815.781831851] [bt_navigator]: Activating
[component_container_isolated-1] [INFO] [1788184815.940390467] [bt_navigator]: Creating bond (bt_navigator) to lifecycle manager.
[component_container_isolated-1] [INFO] [1788184816.049815912] [lifecycle_manager_navigation]: Server bt_navigator connected with bond.
[component_container_isolated-1] [INFO] [1788184816.049956333] [lifecycle_manager_navigation]: Activating waypoint_follower
[component_container_isolated-1] [INFO] [1788184816.051618758] [waypoint_follower]: Activating
[component_container_isolated-1] [INFO] [1788184816.051738666] [waypoint_follower]: Creating bond (waypoint_follower) to lifecycle manager.
[component_container_isolated-1] [INFO] [1788184816.160281168] [lifecycle_manager_navigation]: Server waypoint_follower connected with bond.
[component_container_isolated-1] [INFO] [1788184816.160401300] [lifecycle_manager_navigation]: Activating velocity_smoother
[component_container_isolated-1] [INFO] [1788184816.160840675] [velocity_smoother]: Activating
[component_container_isolated-1] [INFO] [1788184816.160970599] [velocity_smoother]: Creating bond (velocity_smoother) to lifecycle manager.
[component_container_isolated-1] [INFO] [1788184816.268646608] [lifecycle_manager_navigation]: Server velocity_smoother connected with bond.
[component_container_isolated-1] [INFO] [1788184816.268767732] [lifecycle_manager_navigation]: Managed nodes are active
[component_container_isolated-1] [INFO] [1788184816.268822838] [lifecycle_manager_navigation]: Creating bond timer...
```



4. Synchronous Mapping Navigation Operation and Map Save Instructions

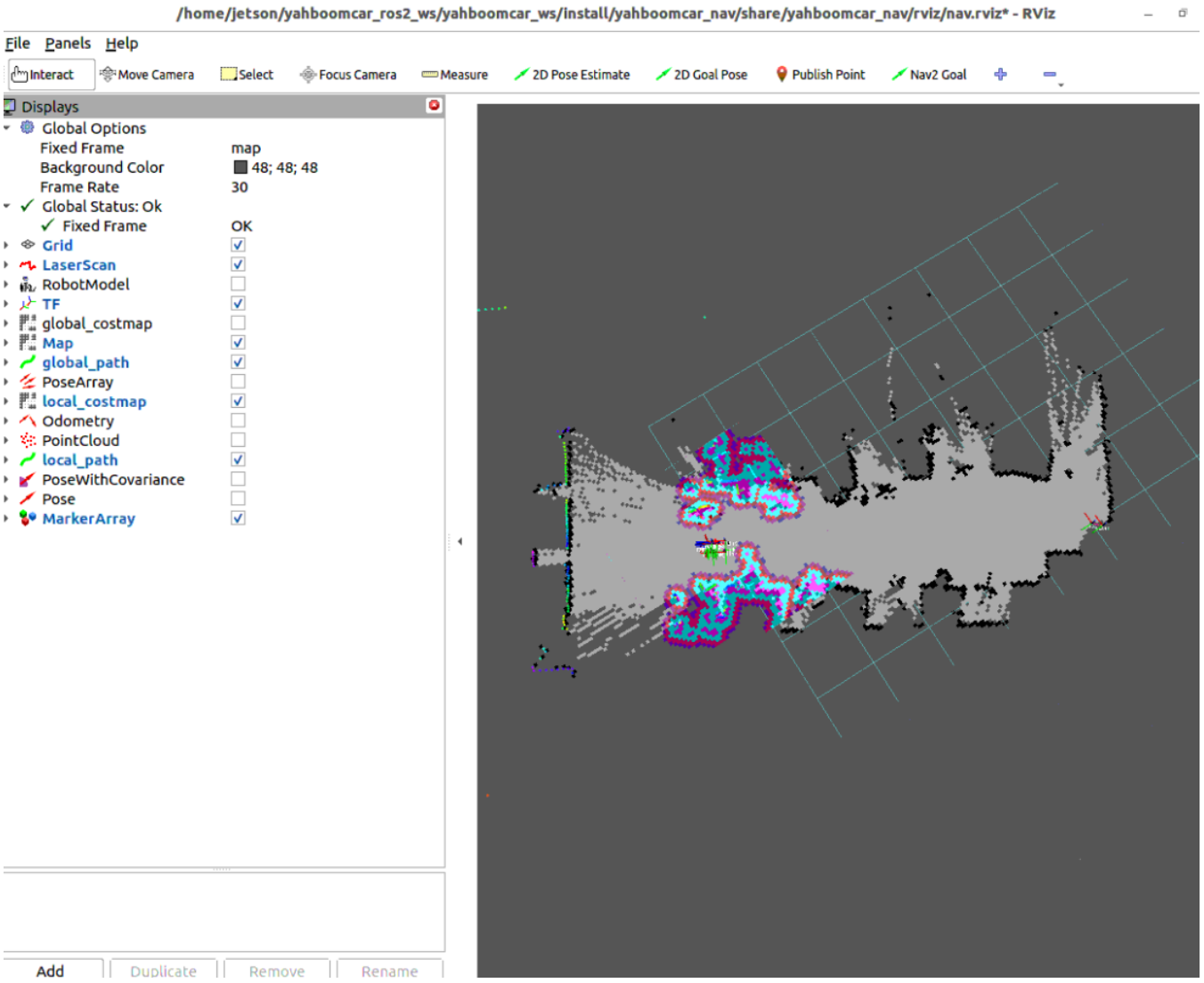
4.1 Send Unknown Area Exploration Goal Point (Nav2 Goal)

1. Click the `Nav2 Goal` (or `2D Goal Pose`) tool in the top toolbar of RViz2;
2. Click in the unexplored unknown gray area (or current field of view edge) and pull out the expected goal heading arrow;
3. The global planner will generate an exploration global path passing through known areas and extending into unknown areas.



4.2 Simultaneous Exploration Mapping and Dynamic Obstacle Avoidance Driving

- The robot autonomously starts Ackermann front-wheel steering and smoothly moves forward, heading toward the set unknown goal point;
- As the robot moves forward, the lidar detection range continuously expands; SLAM algorithm real-time incorporates newly detected walls and traversable areas into the global map;
- When encountering unexplored dynamic or static obstacles, the local TEB controller automatically regenerates detouring trajectory, achieving safe exploration.



4.3 Save Exploration Mapping Results

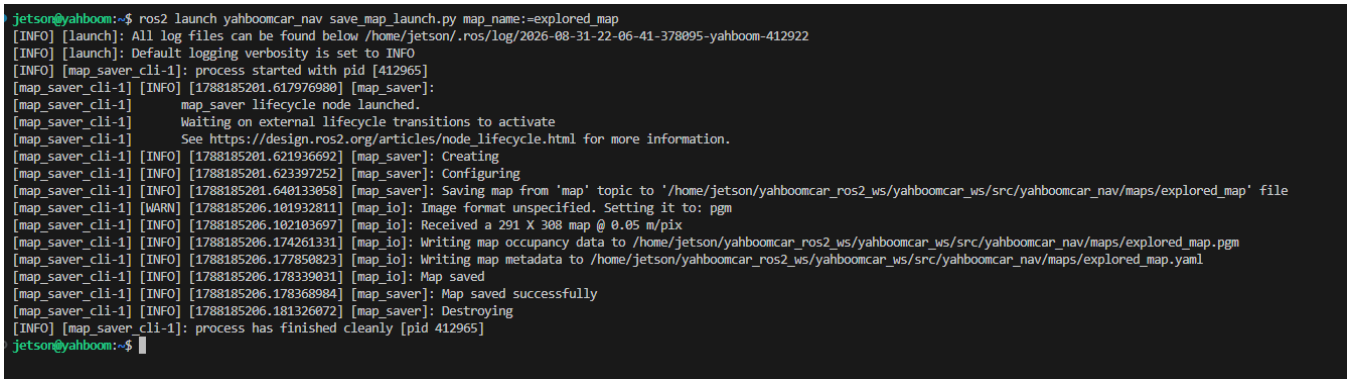
After exploring and completing the target area, run map save commands in **Terminal 4**:

1. **Save 2D Raster Map (.yaml + .pgm)**:

```
ros2 launch yahboomcar_nav save_map_launch.py map_name:=explored_map
```

2. **Save SLAM Pose Graph (.posegraph + .data)**:

```
ros2 launch yahboomcar_nav save_map_launch.py map_type:=posegraph
map_name:=explored_map
```



> **Note**
>
> After exploration mapping and saving are complete, press `Ctrl+C` in each terminal window to stop the program.

5. Core Source Code and Linkage Mechanism Analysis

5.1 Launch File and SLAM/Nav2 Bridge Analysis

File Path:

```
$HOME/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_nav/launch/nav2/navigation_slamtoolx_launch.py
```

Key Source Code and Logic Analysis

```
def generate_launch_description():
    ...
    # Simultaneous mapping and navigation (when slam=True)
    nav_and_slam = GroupAction(
        condition=IfCondition(PythonExpression(["'", slam, "' == 'True'"])),
        actions=[
            PushRosNamespace(namespace=namespace),
            # Introduce Nav2 standard Bringup Launch and pass slam:=True
            IncludeLaunchDescription(
                PythonLaunchDescriptionSource(nav2_launch_file),
                launch_arguments={
                    "map": map_dir,
                    "use_sim_time": use_sim_time,
                    "params_file": params_file,
                    'slam': 'True',
                }.items(),
            ),
            LogInfo(
                msg=[Fore.GREEN+'====================Robot Navigation_SLAM Start (Simultaneous Mapping and Navigation)====================', Fore.RESET]
            )
        ]
    )
```

```
    return LaunchDescription([
        *declared_arguments,
        slamtoolbox_localization_nav,
        nav_and_slam,
    ])
```

- When `slam:=True` is passed, the system directly calls `nav2_bringup`'s `bringup_launch.py` and activates internal `slam_launch.py`, pulling up SLAM-Toolbox asynchronous online mapping;
- At this time, no offline map file needs to be pre-loaded; the SLAM node directly serves as the map publishing source (publishing map to `/map` topic) and global localization provider (publishing `map -> odom` TF transformation).

5.2 Core Navigation and Costmap Parameter Interpretation

Parameter Configuration File Path:

```
$HOME/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_nav/params/teb_nav_params.yaml
```

Core Parameter Description Table

Parameter Module	Parameter Name	Current Config Value	Core Function and Physical Meaning in Simultaneous Mapping Navigation
Startup Mode	`slam`	`True`	Enable simultaneous mapping navigation main switch (automatically start SLAM and bridge Nav2)
Global Planner	`allow_unknown`	`true`	**Core key parameter**: Allow planner to pass through unknown areas to generate exploration paths
Global Costmap	`map_subscribe_transient_local`	`true`	Allow costmap to continuously receive SLAM node dynamic updates and publish latest map
Global Costmap	`subscribe_to_updates`	`true`	Enable incremental map update subscription, reducing full map transmission bandwidth
Local Costmap	`rolling_window`	`true`	Local costmap real-time rolling update following robot pose
Local Controller	`FollowPath.plugin`	`TebLocalPlannerROS`	TEB local time-optimal trajectory optimization planner
Local Controller	`max_vel_x`	`0.25`	Exploration cruising maximum forward linear velocity (m/s)
Local Controller	`min_vel_x`	`-0.05`	Strictly limit reverse speed, preventing rear blind zone collision

Parameter Module	Parameter Name	Current Config Value	Core Function and Physical Meaning in Simultaneous Mapping Navigation
Local Controller	`min_obstacle_dist`	`0.1`	Minimum safe obstacle avoidance gap between robot outline and obstacles (meters)
Local Costmap	`inflation_radius`	`0.18`	Obstacle dynamic inflation radius (meters)

Configuration File Core Snippet with Inline Comments

```yaml
# 1. Global planner configuration: Allow exploration of unknown areas
planner_server:
  ros__parameters:
    expected_planner_frequency: 20.0
    use_sim_time: False
    planner_plugins: ["GridBased"]
    GridBased:
      plugin: "nav2_navfn_planner/NavfnPlanner"
      tolerance: 0.5
      use_astar: false
      allow_unknown: true          # Must be set to true, otherwise cannot plan paths to
unknown areas

# 2. Global costmap configuration: Dynamically subscribe to SLAM online map
global_costmap:
  global_costmap:
    ros__parameters:
      global_frame: map
      robot_base_frame: base_footprint
      rolling_window: false
      resolution: 0.05
      plugins: ["static_layer", "obstacle_layer", "voxel_layer", "inflation_layer"]
      static_layer:
        plugin: "nav2_costmap_2d::StaticLayer"
        map_subscribe_transient_local: True
        enabled: true
        subscribe_to_updates: true # Real-time receive SLAM map incremental updates

# 3. Local TEB obstacle avoidance and trajectory optimization configuration
controller_server:
  ros__parameters:
    FollowPath:
      plugin: "teb_local_planner::TebLocalPlannerROS"
      max_vel_x: 0.25
      min_vel_x: -0.05
      max_vel_theta: 1.0
      min_obstacle_dist: 0.1
      xy_goal_tolerance: 0.25
      yaw_goal_tolerance: 0.25
```